Software Engineering Assignment #03

Smart Campus Room Availability Checker

Boranov Daniyar | Student ID: 24B031718
Class: Thursday 14:00–17:00 | 23 October 2026

1. Prototype Access

Bolt.new project: https://bolt.new/~/sb1-bskzikuu

Public live demo: https://bdaniyar.github.io/boltnew3assSE/

GitHub repository: https://github.com/bdaniyar/boltnew3assSE

Public access and the latest version were verified. Mandatory published-page cases passed: 6. Checked: 2026-10-23T21:43:24.234769+05:00.

The GitHub repository is private; the complete source is included in the submission package.

2. Project overview and approved scope

The goal of this project is to create a simple Smart Campus Room Availability Checker. The system allows students to check whether a selected university room is available at a specific time.

Students can choose a start time and an end time. The system checks the time limits, room status, and existing bookings. After checking, it displays either "Available" or "Unavailable" with a clear explanation.

The application will have one responsive web page and use fictional booking data. It will only check availability and will not create or confirm bookings.

Project baseline

This Assignment #03 continues the existing Bolt.new project. The original React and TypeScript application was preserved before cleanup. The fixed demonstration data is room A-102, now=540 (09:00), blocked=False and one existing interval 600–660 (10:00–11:00).

Original commit: 687c5f7bbcb29590ad3030cf5e596431be5971b7. Recoverable tag: prototype-v1. The source archive and original files are retained under evidence/original-v1/.

The project has no login, database, backend, external API, payments or booking confirmation. All room and booking data is fictional.

3. Initial assumptions and failure points

Initial assumptions

• The system works with one selected room.

• All times are represented as integer minutes after midnight.

• The current time (`now`) is provided and is valid (0–1439).

• The room can be available or blocked.

• Existing bookings are provided as time intervals.

• Students cannot request a time that starts now or in the past.

• The maximum allowed duration is 120 minutes.

• Existing booking data must not be changed.

• No login, database, backend, external API, or payment system is required.

• The interface must work on desktop and mobile devices.

Possible failure points

Issue 1 — Overlapping time intervals

The system might incorrectly reject a request when its start time is exactly equal to the end time of an existing booking. These intervals should not overlap.

Issue 2 — Starting at the current time

The system might accept a request where `start = now`. This should be rejected because the start time must be strictly after the current time.

4. Function contract and initial manual plan

can_book(start, end, now, blocked, existing)

The function receives a requested start and end time, the current time, the room's blocked status, and a list of existing bookings.

It returns True only if all booking conditions are satisfied. Otherwise, it returns False.

The function must not modify the input data.

Initial manual verification plan

I will check the function using valid and invalid time intervals. I will test overlapping bookings, touching endpoints, blocked rooms, the 120-minute limit, and requests starting at the current time. I will also check that the interface displays clear results and works on narrow screens.

Plan review

The cleanup plan preserves v1, keeps AC1–AC6 fixed, removes booking editing, clears stale results, fixes the Start picker and verifies Python, frontend and mobile behavior. The recorded response to this plan was: “Approve this plan”.

The requirements above retain the supplied baseline. The exact plan-review question and reply are preserved in evidence/ai-exchanges/personal-plan-review.json.

5. Acceptance criteria

ID	Fixed requirement
AC1	Integer start/end; 0 <= start < end <= 1440; start > now. Assume now is valid in 0–1439.
AC2	Maximum duration 120 minutes. Exactly 120 is valid; 121 is invalid.
AC3	Reject every request when the selected room is blocked.
AC4	Reject overlap with any existing booking. Half-open intervals may touch at either endpoint.
AC5	Return/display Available or Unavailable; do not create, confirm, delete, modify or mutate bookings.
AC6	Give clear validation feedback and a usable narrow/mobile layout without horizontal overflow.

6. Python implementation overview

booking.py defines can_book(start, end, now, blocked, existing). It returns a built-in Boolean and uses only Python 3 features. The first generated file was saved as booking-v1.py before testing. All 47 test methods passed for v1 and final; no Python repair was needed.

Check	Reason and boundary
Exact integer types	AC1: reject float, string, None and bool before arithmetic.
Bounds and ordering	AC1: reject negative, equal, reversed and out-of-day intervals; permit end=1440.
start <= now	AC1: reject the past and equality; a start one minute later can pass.
end - start > 120	AC2: reject 121; permit 120 when other rules pass.
blocked	AC3: reject a blocked room.
Loop and strict overlap	AC4: start < booked_end and end > booked_start; check later bookings too.
Final True	AC5: only reached after every check; read arguments without changing them.

The complete branch review is in evidence/python-review.md. Valid now, Boolean blocked and well-formed existing intervals are contract assumptions; malformed existing data is outside the required scope.

7. Existing Bolt.new prototype

The existing stack was retained: React 18, TypeScript, Vite, Tailwind CSS and Lucide icons. No replacement application or broad redesign was created. The one-room fixture, component structure and six examples remain.

File or component	Role in the final prototype
src/main.tsx / App.tsx	Mount React and coordinate selected times, blocked state and the latest result.
src/utils/availability.ts	Validate AC1–AC4 and return a clear status/message without changing inputs.
src/utils/time.ts	Format integer minutes as HH:MM; preserve the 24:00 end boundary.
src/data/sampleData.ts	Store the fixed fictional room, demo time, frozen bookings and example cases.
TimeControls / RoomCard	Hour/minute selects; blocked-room toggle. Start 00–23; End may use 24:00.
BookingsSection	Display the fictional booking list as read-only.
ResultPanel / ExampleCases	Present Available/Unavailable and reasons; load the six fixture scenarios.
Header / CurrentTimeBadge	Identify the tool and clearly label 09:00 as a fixed demo time.

8. Workflow, preservation and cleanup

Plan: retain the supplied scope and boundary checks. Generate: preserve Python v1 and the original Bolt source. Critique: compare every rule with source and browser behavior. Repair: make focused changes for three reproduced issues. Verify: rerun the complete tests and inspect real desktop/mobile output.

Before editing, the original Git hash was recorded, the prototype-v1 tag was created, and the source was archived. No original screenshots were present in the repository, so v1 was served from the preserved version and genuine before screenshots were captured. The original Bolt prompt and rendered response were recovered from the project chat.

Cleanup removed add/delete booking paths, cleared stale results on time/status changes, fixed Start hour choices and added polite status semantics. Unused Supabase and a missing favicon reference were removed. Compatible lockfile updates reduced dependency advisories without changing the application rules.

The independent archive check compared all 28 archived files with the original tag and found no mismatch. The first Python implementation also remains byte-identical to final.

9. Executed testing results

Both implementations were executed, not only read. The same Python suite and the same frontend validator suite were used for v1 and final. Browser checks exercised the actual rendered controls and status panels.

Test group	Original v1	Final
Python unittest	47/47; exit 0	47/47; exit 0
Actual frontend validator	37/37; exit 0	37/37; exit 0
Browser mandatory fixtures	6/6	6/6
Browser additional boundaries	10/10	10/10
Browser repair checks	3 defects reproduced	8/8 checks pass
Responsive layout	320 px: no overflow	320/375/768/1440 px: 4/4
Lint / typecheck / build	All exited 0	All exited 0

Six mandatory scenarios

Base: now=540, blocked=False, existing=[(600,660)]. Case 3 sets blocked=True. True means Available; False means Unavailable.

Request	Expected	Python v1 / final	Browser v1 / final
660–720	True	True / True	Available / Available
630–690	False	False / False	Unavailable / Unavailable
660–720, blocked	False	False / False	Unavailable / Unavailable
720–840 (120 min)	True	True / True	Available / Available
720–841 (121 min)	False	False / False	Unavailable / Unavailable
540–570 (starts now)	False	False / False	Unavailable / Unavailable

Boundary and preservation checks

Additional cases covered reversed/equal times, negative and out-of-day values, invalid types, both touching endpoints, containment, overlap in later/unsorted bookings, the last day minute and end=1440. Frozen or nested input data remained unchanged. Browser checks confirmed invalid-time reasons and no controls outside the viewport.

The 37 frontend logic cases also passed after each of the three focused repairs and after compatible dependency updates. A separate review reran Python 47/47 and frontend 37/37. These results do not claim screen-reader testing or live-site deployment unless listed separately.

10. Acceptance traceability

V1 and final actual columns use executed evidence. Passing validator cases in v1 did not establish that the whole original page met AC5 or AC6.

AC ID	Implementation/file	Test input	Expected	V1 actual	Final actual
AC1	booking.py; availability.ts	540–570; reversed/equal; bad type/day bound	Reject	Reject; suites pass	Reject; suites pass
AC1	TimeControls.tsx	11:30, then Start hour 24	No invalid Start option	1470 min; label 24:00	Start only 00–23
AC2	booking.py; availability.ts	720–840 / 720–841	True / False	True / False	True / False
AC3	Both validators; RoomCard	660–720; blocked=True	Unavailable	Fresh check rejects	Rejects; old result clears
AC4	Both validators	660–720 and 550–600; existing 600–660	Both Available	Both Available	Both Available
AC4	Both validators	630–690; later-booking overlap	Reject all overlap	Rejected	Rejected
AC5	App; BookingsSection; data	Delete base booking; repeat checks	Read-only bookings	Delete changes list	No edit controls; unchanged
AC5	Both validators	Available/rejected calls; frozen/nested inputs	Boolean/status; no mutation	No validator mutation	No validator mutation
AC6	App; ResultPanel	Check Available, toggle blocked; invalid start	Clear, current feedback	Stale Available remains	Clears; reason on check
AC6	Responsive layout	V1:320; final:320,375,768,1440 px	No overflow; usable controls	320 px passes	All 4 widths pass

Evidence map

Python: python-v1.* / python-final.* and python-review.md. Validator: frontend-v1-logic.tap / frontend-final-test.tap and after-*-repair.tap. Browser: browser-{v1,final}-{mandatory,boundaries}.json, browser-v1-issues.json, browser-v1-picker.txt, browser-final-repair-checks.json and browser-final-layout.json. All are under evidence/test-results/ except the review note.

Screenshots document the visible state; JSON/TAP/unittest records provide the full executed cases and raw output. The original source remains available for reproducing the before behavior.

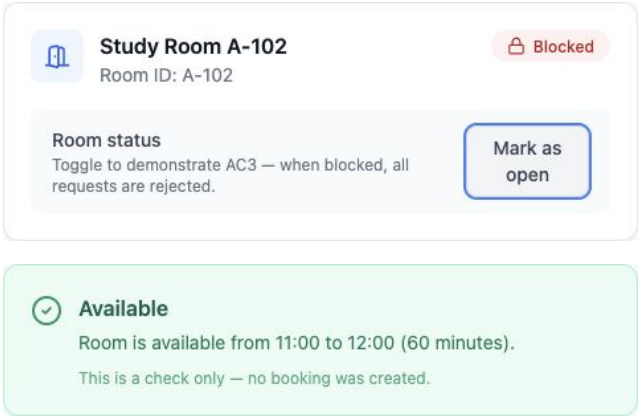
11. Issue log

Issue	Reproduction	Priority	Prompt/change	Retest result
F01: AC5 booking editing	V1: remove 10:00–11:00; list becomes empty. Add/remove controls exist.	High	Keep fixture read-only; remove controls/handlers; freeze data and use readonly types.	No management buttons; list unchanged across cases; logic 37/37.
F02: stale result	Check 11:00–12:00, then mark room blocked. Old Available remains.	High	Clear result after either time or blocked status changes; add polite live status.	All 3 changes clear result; blocked recheck rejects; logic 37/37.
F03: Start hour 24	Set Start 11:30, then hour 24. State=1470; display=24:00.	Medium	Start 00–23 only; End 24:00 forces minute zero; retain validator.	Start option fixed; End minute 0; 23:59–24:00 valid; logic 37/37.

Focused prompt text and complete repair response are preserved in exchange 04; source critique and browser reproduction records establish the original findings. No defect was introduced to create evidence.

12. Before and after: stale blocked result

BEFORE: v1 room blocked



AFTER: final room blocked

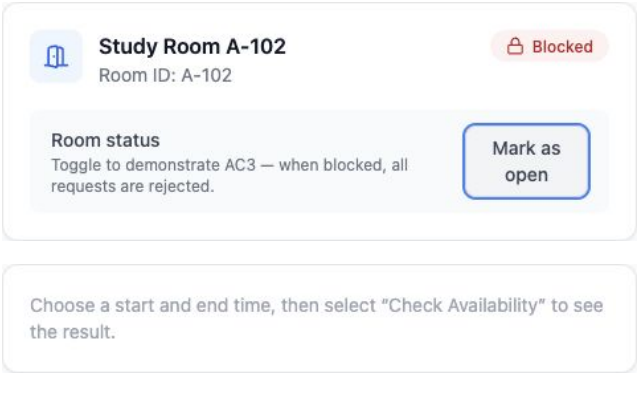


Figure 1. Genuine browser detail captures at the same 1440 px viewport. After an Available check, the room is marked blocked. V1 retains Available; final clears the previous result and asks for a new check. Full original screenshots and native detail captures are retained in evidence/screenshots/.

Preventive cleanup removed an unused dependency and a missing favicon reference. Compatible lockfile repairs reduced the recorded audit from 13 advisories to the final count reported in section 16; application rules and layout were retained.

13. Final prototype evidence

These are genuine browser captures. Details show only selected original screenshot regions; full files preserve the complete context.

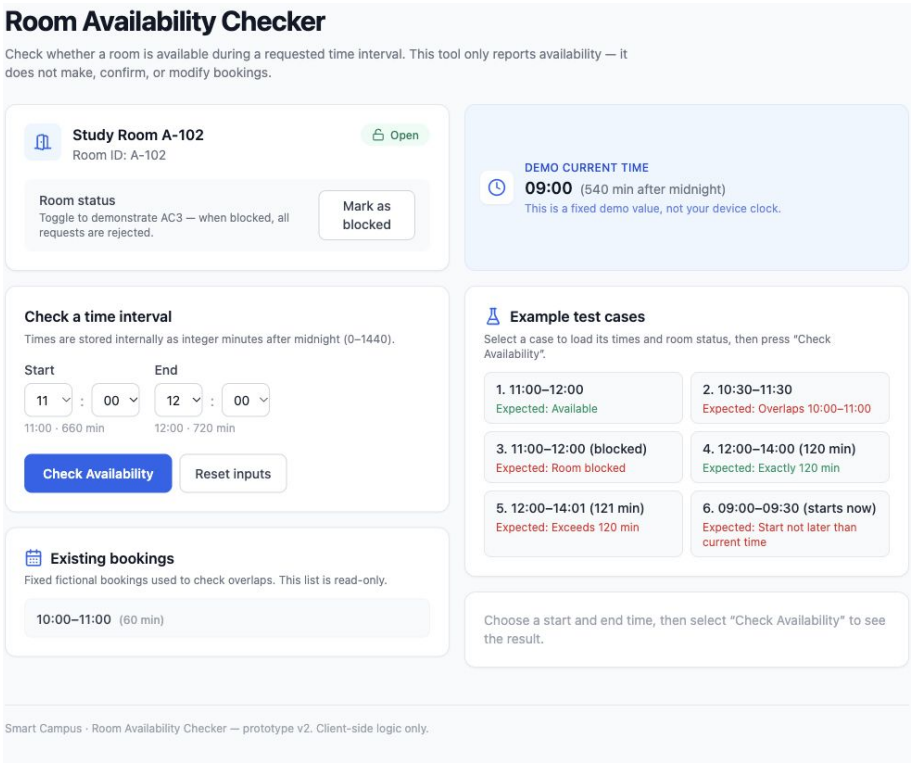


Figure 2. Final desktop interface; original 1440 px capture.

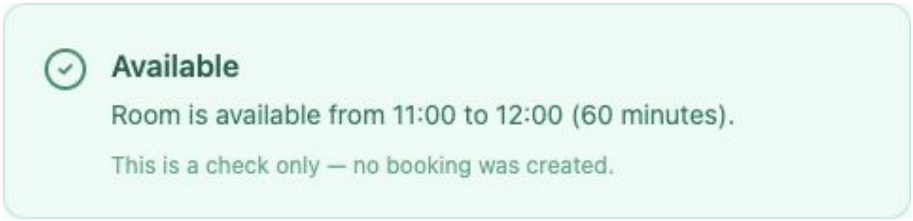


Figure 3. Available: endpoint touch, 660–720.



Figure 4. Unavailable: overlap, 630–690.

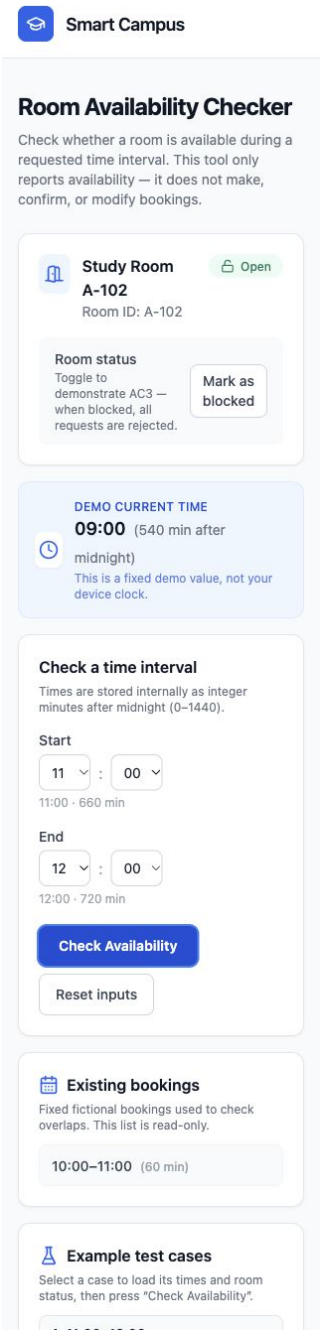


Figure 5. 320 px layout detail. Full-height capture is included in evidence.



Figure 6. Blocked request is Unavailable.



Figure 7. Starts-now validation explains the strict future rule.

All final tested widths (320, 375, 768 and 1440 px) had no horizontal overflow or controls outside the viewport. The availability flow worked at each width. Full Available, overlap, blocked, invalid, 120/121-minute, desktop and mobile screenshots are included in the evidence directory.

14. Coding-agent analysis

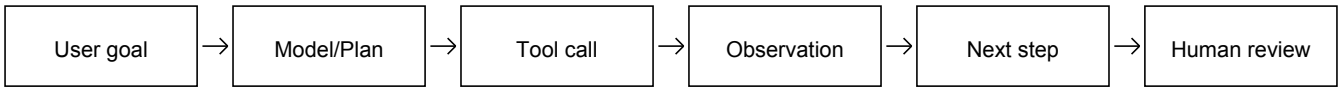


Figure 8. Observable coding-agent loop. An observation may lead to another tool call and test before the result is accepted.

Context, tools and permissions

The assistants used the assignment PDF, supplied requirements, existing repository, Bolt chat, source code, tests and browser observations. Observed tools included shell and file operations, Git, Python unittest, Node's test runner, npm quality checks, browser controls and PDF generation/rendering. The workspace permitted file changes, process execution and network access. Git and GitHub REST published the build using the existing repository credential helper. The Bolt browser workspace was blocked by a startup failure.

Observed behavior and limits

The evidence shows files read and written, commands executed, returned output, screenshots and recorded results. It does not show hidden model reasoning or internal Bolt architecture. Bolt displayed Standard mode, not a model identifier. The exact model names were not displayed for the recorded exchanges.

Safety checkpoint and responsibility

Before generated changes were accepted, the baseline source was preserved. The approved plan was checked against the fixed criteria, and execution records support acceptance. Read-only source review and real tests separated predictions from observed behavior.

Recovery through the original tag supports reliability. Clear current feedback supports usability and availability. Limited permissions, no unnecessary service integration and dependency review reduce security risk. Human review remains responsible for accepting the scope, checking the evidence and deciding what to submit.

15. Reflection

I see AI-assisted coding as a careful process: plan the rules, review the proposed code, and test real results. Vibe coding can produce a useful first page quickly, but a good-looking page can still break a requirement. In this project, the generated add and delete controls were removed because AC5 requires availability checking only. A reliability risk was the old Available result remaining visible after the room became blocked. A usability risk was offering an invalid start hour, while a security risk comes from vulnerable dependencies and broad tool permissions. I understand why code review remains a human responsibility: AI statements about correctness need evidence. Tools should have only the access needed for the task, and changes should be recoverable. The executed tests give stronger support than a successful build alone. I would next improve keyboard and screen-reader testing and address the remaining dependency advisories, while keeping the same one-room scope.

Reflection length: 151 words.

16. Final verification and limitations

Check	Recorded result
Source committed	e6e5da46857e19a2549a07fc38d362c7c66de8cf
Original recovery point	prototype-v1 at 687c5f7bbcb29590ad3030cf5e596431be5971b7
Public sign-in-free access	Verified
Latest deployed version	Verified
Published mandatory flow	6/6 passed
Access check time	2026-10-23T21:43:24.234769+05:00

GitHub main source e6e5da46857e19a2549a07fc38d362c7c66de8cf and the prototype-v1 tag were pushed and verified.

The same production build is public on GitHub Pages. Anonymous requests returned HTTP 200 for HTML, JavaScript and CSS; each matched the local build byte for byte. The published browser passed all six mandatory cases and the 320 px layout check.

Bolt showed a Synced badge, but its latest source could not be inspected after refresh. The startup error stated: WebContainer took longer than 1 minute to boot. Bolt synchronization and Bolt publication remain unverified.

Remaining Bolt action: open the original project in a working Chromium session, load or sync the latest GitHub main source, confirm prototype v2 and no booking editing controls, then Publish and test the Bolt-hosted URL.

The GitHub repository remains private. Complete source is included in this submission package.

Quality and remaining limits

The final Python suite, frontend logic suite, lint, TypeScript check and production build passed. Source, original v1 evidence, actual test logs, screenshots, AI exchanges and the access record are included. The final documents were rendered and inspected for layout; the QA record is under evidence/report-qa/.

The last recorded dependency audit contains 6 advisories (2 low, 3 moderate, 1 high, 0 critical). Compatible transitive updates were applied. The remaining Tailwind/PostCSS chain proposes a Tailwind major-version migration; other residual advisories also remain for later review. A broad upgrade was not forced during this focused cleanup. These counts describe the recorded audit, not a vulnerability-free project.

Keyboard and screen-reader behavior was not fully tested. Polite live-status attributes were verified, but no screen-reader outcome is claimed. The application uses a fixed demo clock and fictional data; it does not check real campus room availability.

References

[1] Software Engineering Assignment #03, supplied 30-page PDF. Scope and criteria: pp. 8–10; planning/Python/prototype/tests: pp. 11–14; cleanup/agent analysis: pp. 15–19; evidence/reflection: pp. 20–23; submission: pp. 28–29.

[2] Lesson #05 and Lesson #06 references listed in the assignment, pp. 25–26. Lecture materials were not separately provided; no additional claims are attributed to them.

[3] Existing Bolt.new project, preserved Git tag prototype-v1, final repository source, and original chat. Access links are on page 1 and repeated in README.md.

[4] Local evidence: Python unittest logs; frontend TAP/quality/audit logs; browser JSON and screenshots; independent integrity review; prototype-access.json; AI_Evidence.pdf and exact exchange text files.

Reproduction

Python: python3 -m unittest -v test_booking. Frontend: npm ci; npm test; npm run lint; npm run typecheck; npm run build. Serve the existing source with npm run dev. Full setup, source recovery and exact access notes are in README.md.